Module 2

Class Fundamentals, Objects creation, Reference Variables and Assignment, Methods, Returning a Value, Using Parameters, Constructors, Parameterized Constructors, new Operator, this Keyword, finalize() method, Wrapper Classes, Parsing, Auto boxing and Unboxing. I/O: Command-Line Arguments, Scanner and Buffered Reader Classes,

A Closer Look into Methods and Classes: Controlling Access to Class Members, passing objects to methods, passing arguments, Returning Objects, Method Overloading, Overloading Constructors, Understanding Static, Variable-Length Arguments.

Creating an Object: new Keyword

Syntax :

ClassName ReferenceVariable = new ClassName();

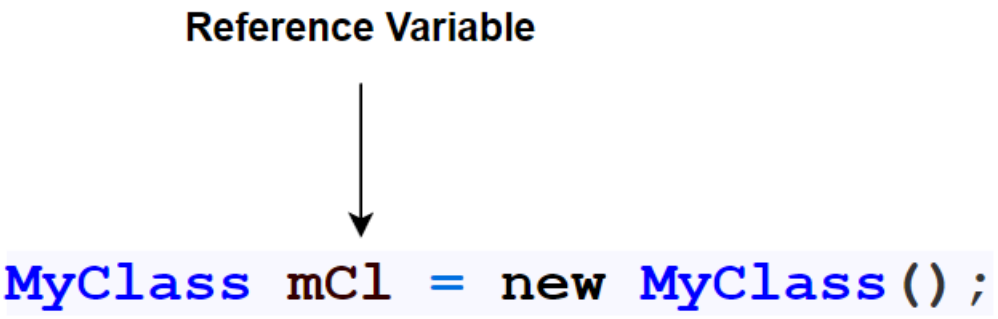
```
public class NewDemo
{
    void display (){
        System.out.println ("Invoking Method");
    }

    public static void main (String[]args)    {
        NewDemo obj = new NewDemo ();
        obj.display ();
    }
}
```

Reference Variables

- ✓ Variable is a name that is used to hold a value of any type during program execution.
  - if the type is an object, then the variable is called reference variable.
  - if the variable holds primitive types(int, float, etc.), then it is called non-reference variables.
- ✓ A variable that holds reference of an object is called a reference variable.

Reference Variables

```
class MyClass{
    void showMyName(String name) {
        System.out.println("Your name is "+name);
    }
}
public class Demo {
    public static void main(String[] args){
        // Creating reference to hold object of MyClass
        MyClass mC1 = new MyClass();
        mC1.showMyName("Java");
    }
}
```



Reference Variables

```
String str = "Java"; // str is reference variable
MyClass mC1 = new MyClass(); // mCl is reference variable
int a = 10; // non-reference variable
```

Reference variable can be of many types based on their scope and accessibility

- Static reference variable
- Instance reference variable
- Local reference variable

Reference Variables

Static reference variable

- A variable defined using static keyword is called static variable.
- Static variable can be a reference or non-reference as well.

```
public class Demo {
    // Static reference variables
     static String str;
    static String str2 = "Java";
   public static void main(String[] args){
        System.out.println("Value of str : "+str);
        System.out.println("Value of str2 : "+str2);
    }
}
```

Reference Variables

Instance reference variable

A variable which is not static and defined inside a class is called an instance reference variable. Since instance variable belongs to object, then we need to use reference variable to call instance variable.

```
public class Demo {
    // instance reference variables
    String str;
    String str2 = "Java";
     public static void main(String[] args){
        Demo demo = new Demo();
        System.out.println("Value of str : "+demo.str
);
        System.out.println("Value of str2 : "+demo.s
tr2);
    }
```

Reference Variables

Local reference variable

Reference variable can be declared as local variable. For example, we created a reference of String class in main() method as local reference variable.

```
public class Demo {
    public static void main(String[] args){
      // Local reference variables
      String str2 = "Java";
      System.out.println("Value of str2 : "+st
r2);
    }
}
```

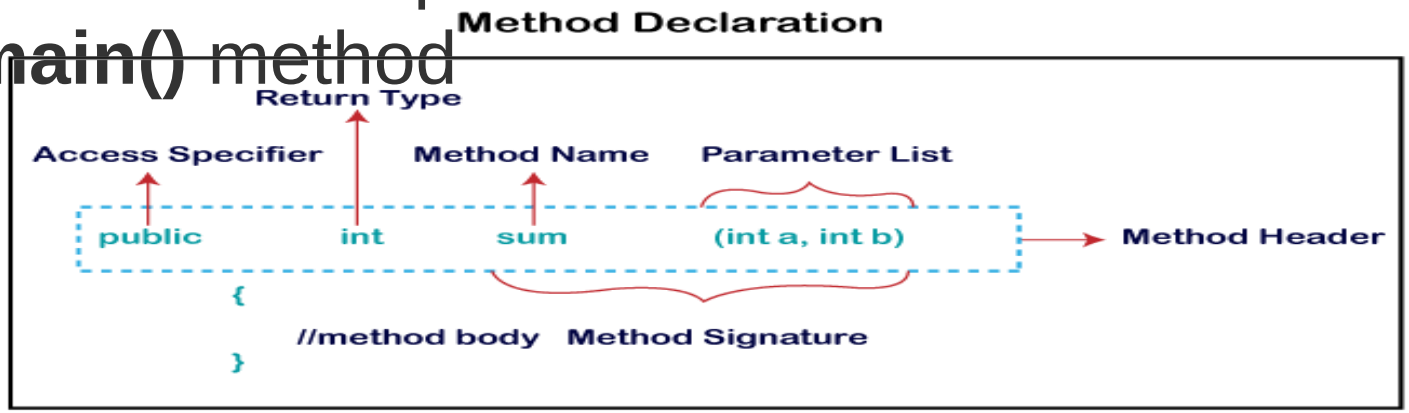
Methods

- A method is a way to perform some task/

A **method** is a block of code or collection of statements or a set of code grouped together to perform a certain task or operation

It is used to achieve the **reusability** of code

The most important method in Java is the **main()** method




```
public int sum(int a,int b){
    //method body
}
```

Types of Method

There are two types of methods in Java:

- Predefined Method(**standard library method** or **built-in method**)
- User-defined Method

The method written by the user or programmer is known as a user-defined method. These methods are modified according to the requirement.

```
//user defined method
public static void findEvenOdd(int num
)
  {  //method body
if(num%2==0)
System.out.println(num+" is even");
else
System.out.println(num+" is odd");
}
```

```
import java.util.Scanner;
public class EvenOdd
{ public static void main (String args[])
    {    //creating Scanner class object
      Scanner scan=new Scanner(System.
      System.out.print("Enter the number: "
       //reading value from the user
     int num=scan.nextInt();
       //method calling
     findEvenOdd(num);
}
```

Types of Method

```
public class Addition
{
public static void main(String[] args)
{ int a = 19;  int b = 5;

int c = add(a, b);   //method calling  a and b are actual parameters
System.out.println("The sum of a and b is= " + c);
}
//user defined method
public static int add(int n1, int n2)   //n1 and n2 are formal parameters
{
int s;
s=n1+n2;
return s; //returning the sum
}
}
```

Static Method

A method that has static keyword is known as static method
a static method is that we can call it without creating an object

```
public class Display
{
        public static void main(String[] args)  {
            show();
          }
        static void show()     {

System.out.println("It is an example of static method.");
        }
}
```

Constructors

In Java, a constructor is a block of codes similar to the method. It is called when an instance of the class is created. At the time of calling constructor, memory for the object is allocated in the memory.

Rules for creating Java constructor

There are two rules defined for the constructor.

- Constructor name must be the same as its class name
- A Constructor must have no explicit return type
- A Java constructor cannot be abstract, static, final, and synchronized

Constructors

Types of Java constructors

There are two types of constructors in Java:

- Default constructor (no-arg constructor)
- Parameterized constructor

```
class Bike1{
//creating a default constructor
Bike1(){System.out.println("Bike is created");}
//main method
public static void main(String args[]){
//calling a default constructor
Bike1 b=new Bike1();
}
}
```

Constructors

```
class Student3{
int id;
String name;
//method to display the value of id and name
void display(){System.out.println(id+" "+name);}
public static void main(String args[]){
//creating objects
Student3 s1=new Student3();
Student3 s2=new Student3();
//displaying values of the object
s1.display();
s2.display();
}
}
```

In the above class,you are not creating any constructor so compiler provides you a default constructor. Here 0 and null values are provided by default constructor.

parameterized constructor

```
class Student4{
    int id;
    String name;
    Student4(int i,String n){
    id = i;
    name = n;
    }
    void display(){System.out.println(id+" "+name);}
       public static void main(String args[]){
    //creating objects and passing values
    Student4 s1 = new Student4(111,"Karan");
    Student4 s2 = new Student4(222,"Aryan");
    //calling method to display the values of object
    s1.display();
    s2.display();
   } }
```

Constructor Overloading

```
class Student5{
   int id;
   String name;
    int age;
    //creating two arg constructor
    Student5(int i,String n){
      id = i;
     name = n;
    }
    //creating three arg constructor
    Student5(int i,String n,int a){
    id = i;
    name = n;
    age=a;
    }
    void display(){System.out.println(id+" "+name+" "+age);}
```

Constructor Overloading

```
class Student5{
   int id;
   String name;
    int age;
    //creating two arg constructor
    Student5(int i,String n){
      id = i;
     name = n;
    }
    //creating three arg constructor
    Student5(int i,String n,int a){
    id = i;
    name = n;
    age=a;
    }
```

Constructor Overloading

```
    void display(){System.out.println(id+" "+name+" "+age);}

    public static void main(String args[]){
    Student5 s1 = new Student5(111,"Karan");
    Student5 s2 = new Student5(222,"Aryan",25);
    s1.display();
    s2.display();
   }
}
```

Constructor Overloading

```
//Java program to initialize the values from one object to anothe
r object.
class Student6{
    int id;
    String name;
    //constructor to initialize integer and string
    Student6(int i,String n){
    id = i;
    name = n;
    }
    //constructor to initialize another object
    Student6(Student6 s){
    id = s.id;
    name =s.name;
    }
    void display(){System.out.println(id+" "+name);}
```

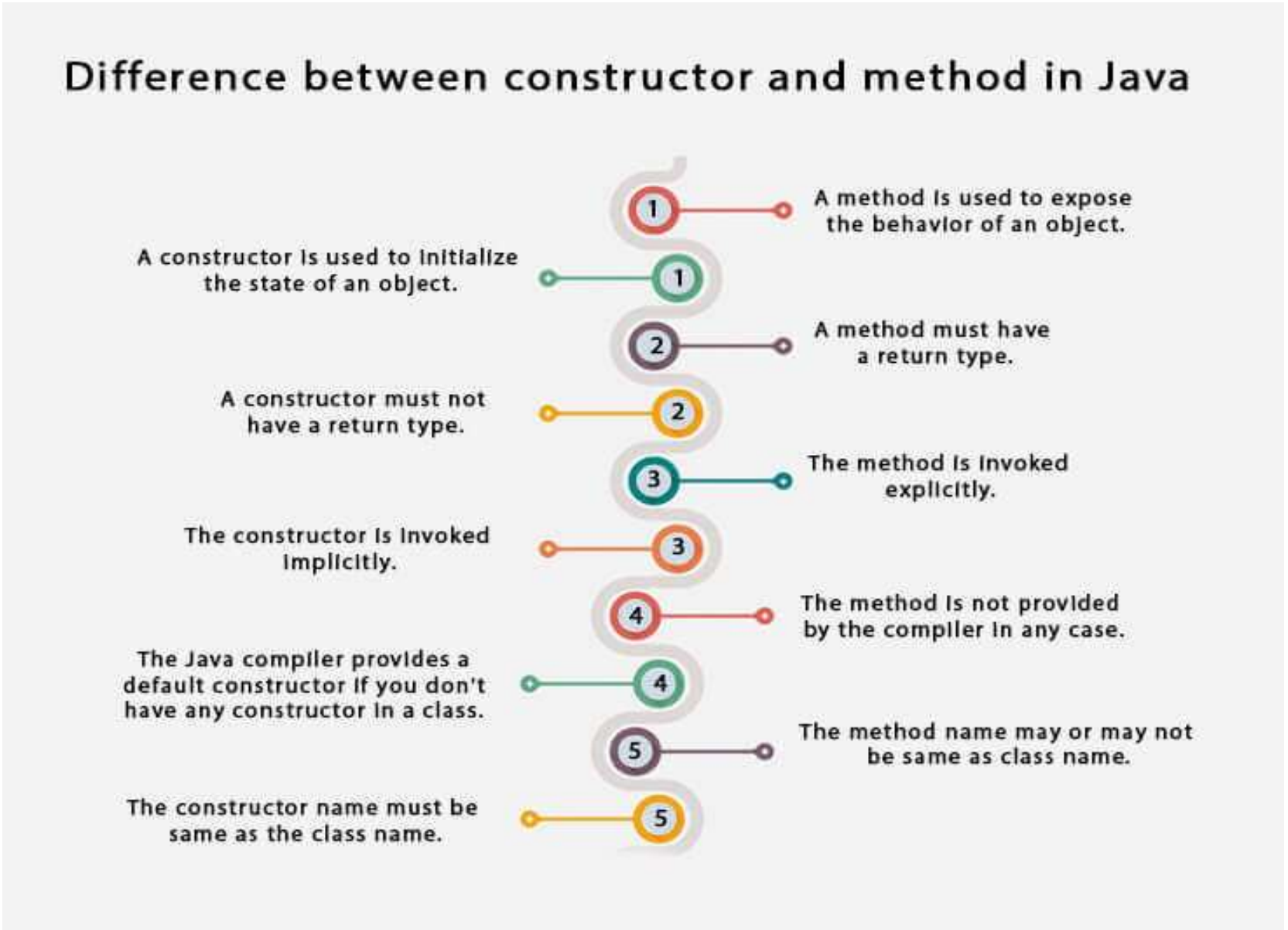
Constructor Overloading

```
    public static void main(String args[]){
    Student6 s1 = new Student6(111,"Karan");
    Student6 s2 = new Student6(s1);
    s1.display();
    s2.display();
   }
}
```

Constructor Overloading

```
class Student7{
    int id;
    String name;
    Student7(int i,String n){
    id = i;
    name = n;
    }
    Student7(){}
    void display(){System.out.println(id+" "+name);}

    public static void main(String args[]){
    Student7 s1 = new Student7(111,"Karan");
    Student7 s2 = new Student7();
    s2.id=s1.id;
    s2.name=s1.name;
    s1.display();
    s2.display();
  }
}
```

Constructor Overloading



new operator

Syntax

NewExample obj=**new** NewExamp
le();

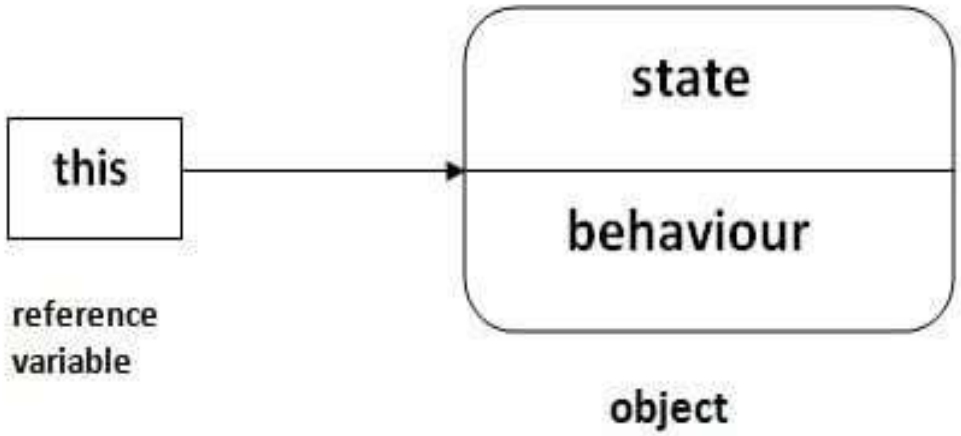
- It is used to create the object.
- It allocates the memory at runtime.
- All objects occupy memory in the heap area.
- It invokes the object constructor.
- It requires a single, postfix argument to call the constructor

new operator

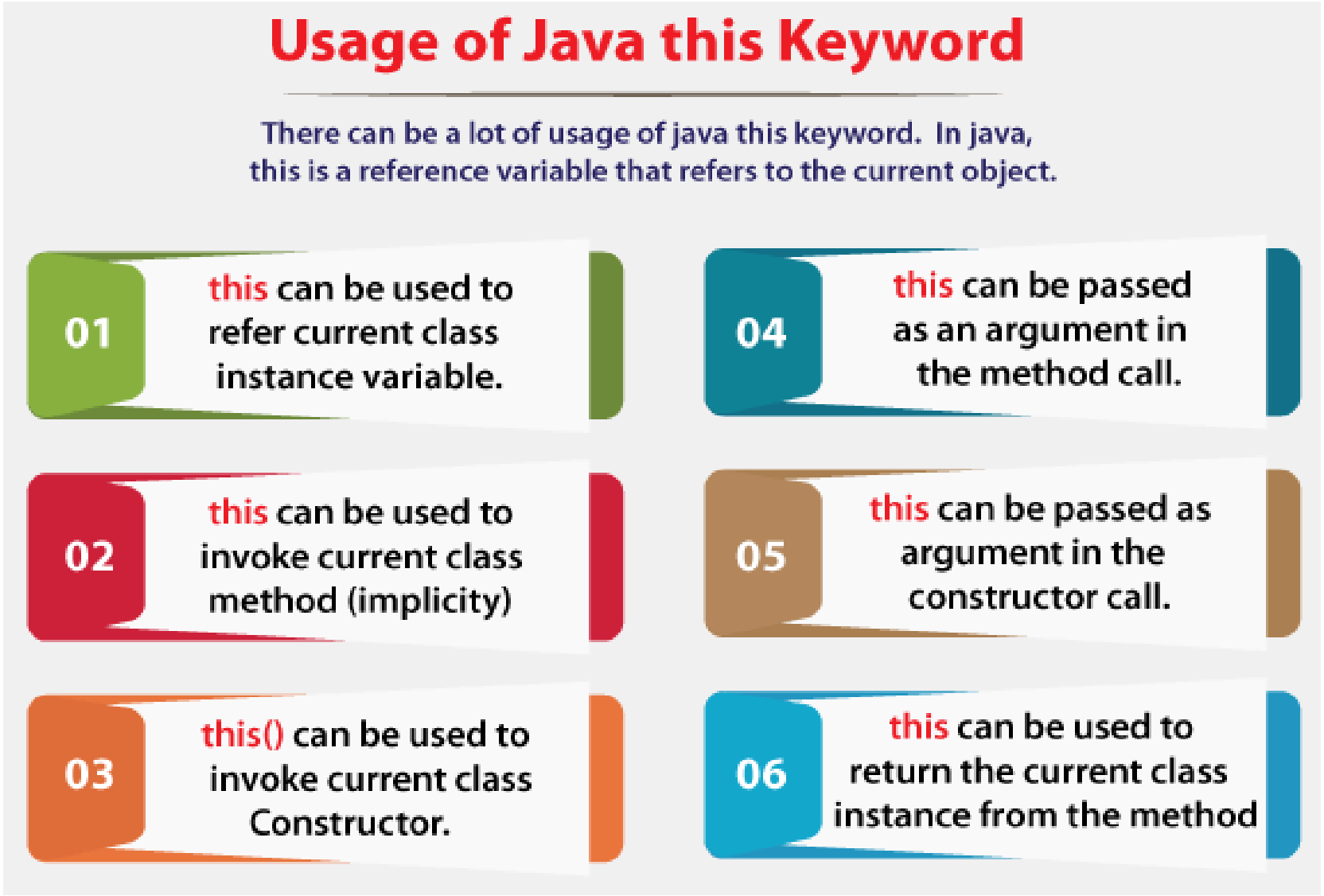
```
public class NewExample2 {
   NewExample2()
    {
        System.out.println("Invoking Constructor");
    }
public static void main(String[] args) {
        NewExample2 obj=new NewExample2();
    }

}
```

this Keyword

this is a reference variable that refers to the current object.



this Keyword



this Keyword

understand the problem if we don't use this keyword

```
class Student{
  int rollno;
  String name;
  float fee;
  Student(int rollno,String name,float fee){
     rollno=rollno;
     name=name;
     fee=fee;
  }
 void display(){System.out.println(rollno+" "+name+" "+fee);}
}
```

this Keyword

understand the problem if we don't use this keyword

```
class TestThis1{
public static void main(String args[]){
Student s1=new Student(111,"ankit",5000f);
Student s2=new Student(112,"sumit",6000f);
s1.display();
s2.display();
}}
```

Output:

```
0 null 0.0
0 null 0.0
```

this Keyword

Solution of the above problem by this keyword

```
class Student{
   int rollno;
   String name;
   Float fee;

Student(int rollno,String name,float fee){
  this.rollno=rollno;
  this.name=name;
  this.fee=fee;
 }
void display(){System.out.println(rollno+" "+nme+" "+fee);}
}
```

this Keyword

Solution of the above problem by this keyword

```
class TestThis2{
public static void main(String args[]){
Student s1=new Student(111,"ankit",5000f);
Student s2=new Student(112,"sumit",6000f);
s1.display();
s2.display();
}}
```

Output:

111 ankit 5000.0
112 sumit 6000.0

finalize() method

- ✓ Finalize() is the method of Object class
- ✓ This method is called just before an object is garbage collected
- ✓ finalize() method in Java is used to release all the resources used by the object before it is deleted/destroyed by the Garbage collector.
- ✓ finalize is not a reserved keyword, it's a method
- ✓ Once the clean-up activity is done by the finalize() method, garbage collector immediately destroys the Java object
- ✓ Java Virtual Machine(JVM) permits invoking of finalize() method only once per object.
- ✓ The garbage collector in Java can be called explicitly using System.gc(); method:
- ✓ System.gc() is a method in Java that invokes garbage collector which will destroy the unreferenced objects.
- ✓ System.gc() calls finalize() method only once for each object

finalize() method

Syntax **protected void finalize() throws Throwable**

```
public class JavafinalizeExample1 {
    public static void main(String[] args)
     {   JavafinalizeExample1 obj = new JavafinalizeExample1(
);
         System.out.println(obj.hashCode());
         obj = null;
         // calling garbage collector
         System.gc();
         System.out.println("end of garbage collection");
     }
    @Override
    protected void finalize()
    {          System.out.println("finalize method called");       }

}
```

Wrapper Classes

The wrapper class in Java provides the mechanism to convert primitive into object and object into primitive

Primitive Data Type	Wrapper Class
char	Character
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
boolean	Boolean

Parsing

```
public class ParseInt0
{ public static void main(String args[])
        {  int x = Integer.parseInt("12");
           double c = Double.parseDouble("12");
           int b = Integer.parseInt("100",2);
           System.out.println(Integer.parseInt("12"));
           System.out.println(Double.parseDouble("12"));
           System.out.println(Integer.parseInt("100",2));
           System.out.println(Integer.parseInt("101", 8));
    }
}
```

Parsing

```
import java.time.Period;
public class ParseDemo1
  { public static void main(String[] args)
      {   //Here is the age String in format to parse
          String age = "P17Y9M5D";
          // Converting strings into period value
          // using parse() method
          Period p = Period.parse(age);
          System.out.println("the age is: ");
          System.out.println(p.getYears() + " Years\n" +
p.getMonths() + " Months\n" + p.getDays() + " Days\n"); }
}
```

Auto boxing and Unboxing

Autoboxing: Automatic conversion of primitive types to the object of their corresponding wrapper classes is known as autoboxing. For example – conversion of int to Integer, long to Long, double to Double etc.

```
// Java program to demonstrate Autoboxing
import java.util.ArrayList;
class Autoboxing
{
    public static void main(String[] args)
    {
        char ch = 'a';
       // Autoboxing- primitive to Character object conversion
        Character a = ch;
        ArrayList<Integer> arrayList = new ArrayList<Integer>();
        // Autoboxing because ArrayList stores only objects
        arrayList.add(25);
        // printing the values from object
        System.out.println(arrayList.get(0));
    }
}
```

Auto boxing and Unboxing

```
class BoxingExample1{
  public static void main(String args[]){
    int a=50;
        Integer a2=new Integer(a);//Boxing

        Integer a3=5;//Boxing

        System.out.println(a2+" "+a3);
 }
}
```

Auto boxing and Arguments

Unboxing: It is just the reverse process of autoboxing. Automatically converting an object of a wrapper class to its corresponding primitive type is known as unboxing. For example – conversion of Integer to int, Long to long, Double to double, etc.

```
class UnboxingExample1{
  public static void main(String args[]){
    Integer i=new Integer(50);
        int a=i;

        System.out.println(a);
 }
}
```

Auto boxing and Arguments

```
// Java program to demonstrate Unboxing
import java.util.ArrayList;
class Unboxing
{
    public static void main(String[] args)
    {   Character ch = 'a';
        // unboxing - Character object to primitive conversion
        char a = ch;
        ArrayList<Integer> arrayList = new ArrayList<Integer>();
        arrayList.add(24);
        // unboxing because get method returns an Integer object
        int num = arrayList.get(0);
        // printing the values from primitive data types
        System.out.println(num);
    }
}
```

Command-Line Arguments

- ✓ Java command-line argument is an argument i.e. passed at the time of running the Java program
- ✓ the arguments passed from the console can be received in the java program and they can be used as input
- ✓ The users can pass the arguments during the execution bypassing the command-line arguments inside the main() method
- ✓ We need to pass the arguments as space-separated values
- ✓ We can pass both strings and primitive data types(int, double, float, char, etc) as command-line arguments.

Command-Line Arguments

```
class CommandLineExample{
public static void main(String args[]){
System.out.println("Your first argument is: "+args[0]);
}
}
```

1.compile by > javac CommandLineExample.java
2.run by > java CommandLineExample GITAM
Output: Your first argument is: GITAM

Command-Line Arguments

printing all the arguments passed from the command-line

```
class A{
public static void main(String args[]){

for(int i=0;i<args.length;i++)
System.out.println(args[i]);
  }
}
```

1.compile by > javac A.java
2.run by > java A RAJ RAM 1 3 abc
3.Output: RAJ
RAM
1
3
abc

Scanner

- ✓ Scanner is a class in java.util package used for obtaining the input of the primitive types like int, double, etc. and strings.

```
import java.util.Scanner;
public class ScannerDemo1
{		public static void main(String[] args)
		{
			Scanner sc = new Scanner(System.in);
			String name = sc.nextLine();
			char gender = sc.next().charAt(0);
			int age = sc.nextInt();
			long mobileNo = sc.nextLong();
			double cgpa = sc.nextDouble();
			System.out.println("Name: "+name);
			System.out.println("Gender: "+gender);
			System.out.println("Age: "+age);
			System.out.println("Mobile Number: "+mobileNo);
			System.out.println("CGPA: "+cgpa);
		}
}
```

Buffered Reader Classes

Java BufferedReader class is used to read the text from a character-based input stream. It can be used to read data line by line by readLine() method. It makes the performance fast.

```
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
class Employee {
    String name;
    int id;
    int age;
    Employee(String name, int age, int id) {
        this.name = name;
        this.age = age;
        this.id = id;
    }
    public void displayDetails() {
        System.out.println("Name: "+this.name);
        System.out.println("Age: "+this.age);
        System.out.println("Id: "+this.id);
    }
}
```

Buffered Reader Classes

```
public class ReadData {
public static void main(String args[]) throws IOException {
BufferedReader reader =new BufferedReader(new InputStreamReader(System.in));
System.out.println("Enter your name: ");
String name = reader.readLine();
System.out.println("Enter your age: ");
int age = Integer.parseInt(reader.readLine());
System.out.println("Enter your Id: ");
int id = Integer.parseInt(reader.readLine());
Employee std = new Employee(name, age, id);
std.displayDetails();
}
}
```

Output

```
Enter your name:
Krishna
Enter your age:
25
Enter your Id:
1233
Name: Krishna
Age: 25
Id: 1233
```

Controlling Access to Class Members

Access Specifier: Access specifier or modifier is the access type of the method. It specifies the visibility of the method. Java provides **four** types of access specifier:

- **Public:** The method is accessible by all classes when we use public specifier in our application.
- **Private:** When we use a private access specifier, the method is accessible only in the classes in which it is defined.
- **Protected:** When we use protected access specifier, the method is accessible within the same package or subclasses in a different package.
- **Default:** When we do not use any access specifier in the method declaration, Java uses default access specifier by default. It is visible only from the same package only.

Controlling Access to Class Members

Access Levels

Specifier	Class	Package	Subclass	World
`private`	Y	N	N	N
no specifier	Y	Y	N	N
`protected`	Y	Y	Y	N
`public`	Y	Y	Y	Y

Controlling Access to Class Members

```
public class Alpha{
    //member variables
    private   int iamprivate = 1;
              int iampackage = 2;  //package access
    protected int iamprotected = 3;
    public    int iampublic = 4;

    //methods
    private void privateMethod() {
        System.out.println("iamprivate Method");
    }
    void packageMethod() { //package access
        System.out.println("iampackage Method");
    }
    protected void protectedMethod() {
        System.out.println("iamprotected Method");
    }
    public void publicMethod() {
        System.out.println("iampublic Method");
    }
```

Controlling Access to Class Members

```
public static void main(String[] args) {
        Alpha a = new Alpha();
        a.privateMethod();      //legal
        a.packageMethod();   //legal
        a.protectedMethod(); //legal
        a.publicMethod();     //legal

        System.out.println("iamprivate: "    + a.iamprivate);    //legal
        System.out.println("iampackage: “  + a.iampackage);    //legal
        System.out.println("iamprotected: "  + a.iamprotected); //legal
        System.out.println("iampublic: “       + a.iampublic);     //legal
    }
}
```

Controlling Access to Class Members

```
class A{
private int data=40;
private void msg(){System.out.println("Hello java");}
}

public class Simple{
public static void main(String args[]){
  A obj=new A();
  System.out.println(obj.data);//Compile Time Error
 obj.msg();//Compile Time Error
  }
}
```

Controlling Access to Class Members

```
class A{
private A(){}//private constructor
void msg(){System.out.println("Hello java");}
}
public class Simple{
  public static void main(String args[]){
    A obj=new A();//Compile Time Error
  }
}
```

Controlling Access to Class Members

```
//save by A.java
package pack;
class A{
 void msg(){System.out.println("Hello");}
}
```

```
//save by B.java
package mypack;
import pack.*;
class B{
    public static void main(String args[]){
        A obj = new A();//Compile Time Erro
        obj.msg();//Compile Time Error
    }
}
```

```
C:\F DATA\java>javac B.java
B.java:2: error: package pack does not exist
import pack.*;
^
B.java:5: error: cannot find symbol
  A obj = new A();//Compile Time Error
  ^
  symbol:   class A
  location: class B
B.java:5: error: cannot find symbol
  A obj = new A();//Compile Time Error
            ^
  symbol:   class A
  location: class B
3 errors
```

Passing and Returning Objects

When we pass a primitive type to a method, it is passed by value. But when we pass an object to a method, the situation changes dramatically, because objects are passed by what is effectively call-by-reference. Java does this interesting thing that's sort of a hybrid between pass-by-value and pass-by-reference.

In case of call by reference original value is changed if we made changes in the called method. If we pass object in place of any primitive value, original value will be changed. In this example we are passing object as a value.

Passing Objects

```
class Operation2{
 int data=50;

 void change(Operation2 op){
    op.data=op.data+100;//changes will be in the instance variabl
e
 }

 public static void main(String args[]){
    Operation2 op=new Operation2();
    System.out.println("before change "+op.data);
    op.change(op);//passing object
    System.out.println("after change "+op.data);

 }
```

Output:before change 50

passing objects to methods

```
class ObjectPassDemo
{
    int a, b;
   ObjectPassDemo(int i, int j)
     {   a = i;  b = j;   }
   boolean equalTo(ObjectPassDemo o)
    {   return (o.a == a && o.b == b);  }
}
public class Test
{ public static void main(String args[])
    {  ObjectPassDemo ob1 = new ObjectPassDemo(100, 22);
       ObjectPassDemo ob2 = new ObjectPassDemo(100, 22);
       ObjectPassDemo ob3 = new ObjectPassDemo(-1, -1);

       System.out.println("ob1 == ob2: " + ob1.equalTo(ob2));
       System.out.println("ob1 == ob3: " + ob1.equalTo(ob3));
     }
}
```

```
C:\F DATA\java>java Test
ob1 == ob2: true
ob1 == ob3: false
```

Returning Objects

```
class Test
{    int a;
     Test(int i)     {    a = i;   }
     Test incrByTen()
      {   Test temp = new Test(a+10);     return temp;   }
}
public class JavaProgram
{       public static void main(String args[])
        {
           Test obj1 = new Test(2);
           Test obj2;
           obj2 = obj1.incrByTen();
           System.out.println("obj1.a : " + obj1.a);
           System.out.println("obj2.a : " + obj2.a);
           obj2 = obj2.incrByTen();
          System.out.println("obj2.a after second increase : " + obj2.a);
        }
}
```

```
C:\F DATA\java>java JavaProgram
obj1.a : 2
obj2.a : 12
obj2.a after second increase : 22
```

Method Overloading

- ✓ If a class has multiple methods having same name but different in parameters, it is known as Method Overloading.

- ✓ Different ways to overload the method

  There are two ways to overload the method in java

  1. By changing number of arguments
  2. By changing the data type

Method Overloading

- ✓ we are creating static methods so that we don't need to create instance for calling methods

```
class Adder{
   static int add(int a,int b){return a+b;}
   static int add(int a,int b,int c){return a+b+c;}
 }

class TestOverloading1{
public static void main(String[] args){
System.out.println(Adder.add(11,11));
System.out.println(Adder.add(11,11,11));
}}
```

Method Overloading

- ✓ methods that differs in data type. The first add method receives two integer arguments and second add method receives two double arguments.

```
class Adder{
static int add(int a, int b){return a+b;}
static double add(double a, double b) {return a+b;}
}

class TestOverloading2{
public static void main(String[] args){
System.out.println(Adder.add(11,11));
System.out.println(Adder.add(12.3,12.6));
}
}
```

Overloading Constructors

```
class Student7{
    int id;      String name;

 Student7(int i,String n){
    id = i;
    name = n;
    }
 Student7(){}
    void display(){System.out.println(id+" "+name);}

    public static void main(String args[]){
    Student7 s1 = new Student7(111,"Karan");
    Student7 s2 = new Student7();
    s2.id=s1.id;
    s2.name=s1.name;
    s1.display();
    s2.display();
  }
}
```

Overloading Constructors

```
class Box
{    double width, height, depth;

  Box(double w, double h, double d)
       {    width = w;          height = h;          depth = d;          }

  Box()
     {   width = height = depth = 0;  }

  Box(double len)
    { width = height = depth = len;}

  double volume()
            {return width * height * depth;}
}
```

Overloading Constructors

```
public class Test
{
        public static void main(String args[])
        {
                Box mybox1 = new Box(10, 20, 15);
                Box mybox2 = new Box();
                Box mycube = new Box(7);

                double vol;
                // get volume of first box
                vol = mybox1.volume();
                System.out.println(" Volume of mybox1 is " + vol);
                 // get volume of second box
                vol = mybox2.volume();
                System.out.println(" Volume of mybox2 is " + vol);
                 // get volume of cube
                vol = mycube.volume();
                System.out.println(" Volume of mycube is " + vol);
        }
}
```

Java static keyword

✓ The static keyword in Java is used for memory management mainly

The static can be:

- Variable (also known as a class variable)
- Method (also known as a class method)
- Block
- Nested class

Java static keyword

static variable

```
class Student{
   int rollno;//instance variable
   String name;
   static String college =“GITAM";//static variable
   //constructor
   Student(int r, String n){
   rollno = r;
   name = n;
   }
   //method to display the values
    void display (){System.out.println(rollno+" "+name+" "+colleg
e);}
}
```

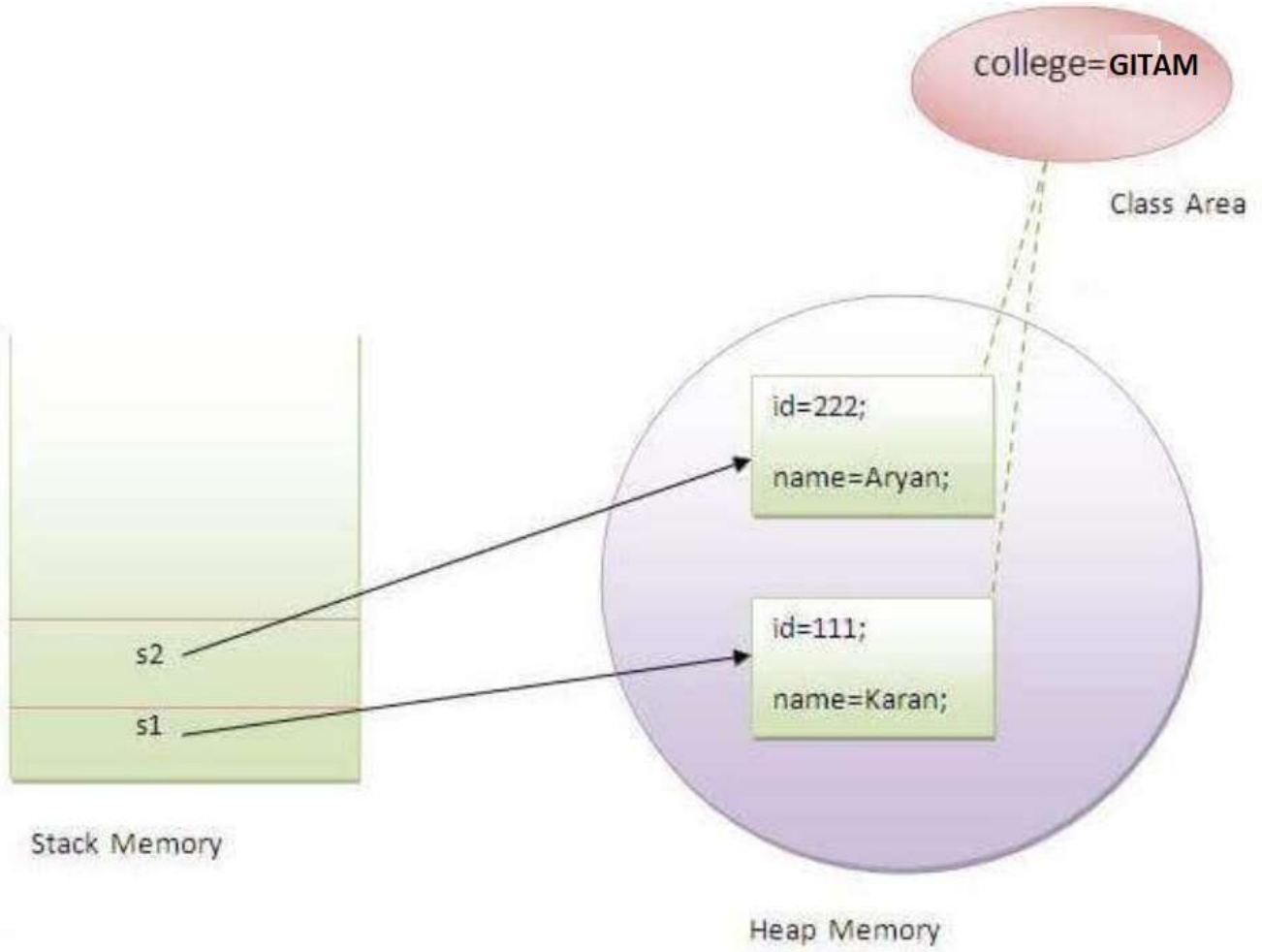
Java static keyword

static variable

```
//Test class to show the values of objects
public class TestStaticVariable1{
 public static void main(String args[]){
 Student s1 = new Student(111,"Karan");
 Student s2 = new Student(222,"Aryan");
 //we can change the college of all objects by the single line of code
 //Student.college="BBDIT";
 s1.display();
 s2.display();
 }
}
```

Java static keyword

static variable



Variable-Length Arguments.

Variable-length argument lists, makes it possible to write a method that accepts any number of arguments when it is called. For example, suppose we need to write a method named sum that can accept any number of int values and then return the sum of those values.

```
public static int sum(int ... list)
{
    //write code here
}
```